

Skill – Week8

Session 15:

1. To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
GNU nano 8.7.1 combined_redirection.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        // Open output file
        int fd = open("combined_output.txt",
                     O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) {
            perror("open");
            exit(1);
        }
        // Redirect stdout to file
        dup2(fd, STDOUT_FILENO);
        // Redirect stderr to same file
        dup2(fd, STDERR_FILENO);
        close(fd);
        // Generate both outputs
        printf("This message is from stdout.\n");
        fprintf(stderr, "This message is from stderr.\n");
        exit(0);
    }
    wait(NULL);
    printf("Combined redirection completed.\n");
    printf("Check combined_output.txt for both streams.\n");
    return 0;
}
```

```

sanjay_dundra@Sanjay:~/OS_Lab$ nano combined_redirection.c
sanjay_dundra@Sanjay:~/OS_Lab$ gcc combined_redirection.c -o combined_redirection
sanjay_dundra@Sanjay:~/OS_Lab$ ./combined_redirection
Combined redirection completed.
Check combined_output.txt for both streams.
sanjay_dundra@Sanjay:~/OS_Lab$ cat combined_output.txt
This message is from stderr.
This message is from stdout.
sanjay_dundra@Sanjay:~/OS_Lab$ ls -l combined_output.txt
-rw-r--r-- 1 sanjay_kumar sanjay_kumar 58 Sep 19 03:11 combined_output.txt
sanjay_dundra@Sanjay:~/OS_Lab$ cat -n combined_output.txt
 1 This message is from stderr.
 2 This message is from stdout.

```

```

sanjay_dundra@Sanjay:~/OS_Lab$ ./combined_redirection
./combined_redirection
cat combined_output.txt
Combined redirection completed.
Check combined_output.txt for both streams.
Combined redirection completed.
Check combined_output.txt for both streams.
This message is from stderr.
This message is from stdout.

```

Report: The program demonstrates combined redirection by using `dup2()` to duplicate the file descriptor for both `stdout` and `stderr`, allowing both streams to be merged into a single file while preserving their write order.

2. To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_ARGS 50
#define MAX_CMDS 10

void parse_command(char *cmd, char **args,
                  char **infile, char **outfile, int *append) {
    char *token;
    int i = 0;

    *infile = NULL;
    *outfile = NULL;
    *append = 0;

    token = strtok(cmd, " \t");

    while (token != NULL && i < MAX_ARGS - 1) {
        if (strcmp(token, "<") == 0) {
            token = strtok(NULL, " \t");
            if (token != NULL)
                *infile = token;
        }
        else if (strcmp(token, ">") == 0) {
            token = strtok(NULL, " \t");
            if (token != NULL) {
                *outfile = token;
                *append = 0;
            }
        }
        else if (strcmp(token, ">>") == 0) {
            token = strtok(NULL, " \t");
        }
    }
}
```

```
sanjay_dundra@Sanjay:~/OS_Lab$ nano complex_command.c
sanjay_dundra@Sanjay:~/OS_Lab$ gcc complex_command.c -o complex_command
sanjay_dundra@Sanjay:~/OS_Lab$ ./complex_command
Enter command: echo "apple" > sample.txt
```

```
Execution Plan:
Command 1: echo "apple" | Output: sample.txt
```

```
Executing...
Command execution completed.
sanjay_dundra@Sanjay:~/OS_Lab$ ./complex_command
Enter command: cat sample.txt | grep apple > result.txt
```

```
Execution Plan:
Command 1: cat sample.txt
Command 2: grep apple | Output: result.txt
```

```
Executing...
Command execution completed.
sanjay_dundra@Sanjay:~/OS_Lab$ cat result.txt
apple
```

```
sanjay_dundra@Sanjay:~/OS_Lab$ ./complex_command
Enter command: cat < sample.txt | grep banana
```

```
Execution Plan:
Command 1: cat | Input: sample.txt
Command 2: grep banana
```

```
Executing...
banana
Command execution completed.
sanjay_dundra@Sanjay:~/OS_Lab$ ./complex_command
Enter command:
Error: Empty command.
sanjay_dundra@Sanjay:~/OS_Lab$ ./complex_command
Enter command: cat sample.txt | grep apple | wc -l
```

```
Execution Plan:
Command 1: cat sample.txt
Command 2: grep apple
Command 3: wc -l
```

```
Executing...
2
Command execution completed.
```

Report: The program parses complex shell commands, separates pipeline stages, handles input/output redirection, creates pipes between processes, validates basic syntax, displays an execution plan, and executes the resulting command pipeline.

Session 16:

1. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```
GNU nano 8.7.1                                background_jobs.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 20

struct Job {
    int job_id;
    pid_t pid;
    char command[100];
};

struct Job jobs[MAX_JOBS];
int job_count = 0;
void check_jobs() {
    for (int i = 0; i < job_count; i++) {
        int status;
        pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);
        if (result == 0) {
            printf("[%d] Running    PID: %d    %s\n",
                   jobs[i].job_id,
                   jobs[i].pid,
                   jobs[i].command);
        }
        else if (result == jobs[i].pid) {
            printf("[%d] Completed PID: %d    %s\n",
                   jobs[i].job_id,
                   jobs[i].pid,
                   jobs[i].command);
            jobs[i].pid = -1;
        }
    }
}

int main() {
    char input[200];
```

```
sanjay_dundra@Sanjay:~/OS_Lab$ nano background_jobs.c
sanjay_dundra@Sanjay:~/OS_Lab$ gcc background_jobs.c -o background_jobs
sanjay_dundra@Sanjay:~/OS_Lab$ ./background_jobs
Background Job Shell
Use '&' to run a command in background.
Use 'jobs' to monitor jobs.
Use 'exit' to quit.

shell> sleep 10 &
[1] 1171 started in background
shell> jobs
[1]  Running      PID: 1171      sleep 10
shell> sleep 15 &
[2] 1204 started in background
shell> sleep 8 &
[3] 1206 started in background
shell> jobs
[1] Completed    PID: 1171      sleep 10
[2] Completed    PID: 1204      sleep 15
[3] Running      PID: 1206      sleep 8
```

Report: The program detects commands ending with &, launches them as background processes without blocking the shell, immediately returns the prompt, stores PID and job information, and monitors job completion using waitpid() with WNOHANG.

2. To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```

GNU nano 8.7.1                                job_table.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 20

typedef enum {
    RUNNING,
    COMPLETED
} JobState;
struct Job {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[100];
    JobState state;
};
struct Job jobs[MAX_JOBS];
int job_count = 0;
/* Display current job table */
void display_jobs() {
    printf("\n%-5s %-8s %-8s %-20s %-10s\n",
           "ID", "PID", "PGID", "COMMAND", "STATE");
    printf("-----\n");
    if (job_count == 0) {
        printf("No active jobs.\n");
        return;
    }
    for (int i = 0; i < job_count; i++) {
        printf("%-5d %-8d %-8d %-20s %-10s\n",
               jobs[i].job_id,
               jobs[i].pid,
               jobs[i].pgid,
               jobs[i].command,
               jobs[i].state);
    }
}

```

```

sanjay_dundra@Sanjay:~/OS_Lab$ nano job_table.c
sanjay_dundra@Sanjay:~/OS_Lab$ gcc job_table.c -o job_table
sanjay_dundra@Sanjay:~/OS_Lab$ ./job_table
Job Table Management System
Commands:
  <command> &    -> Start background job
  jobs           -> Display job table
  update        -> Update job states
  clean         -> Remove completed jobs
  validate      -> Validate job records
  exit          -> Exit

job-shell> sleep 15 &
[1] 1267 PID=1267 PGID=1267 started: sleep 15
job-shell> sleep 8 &
[2] PID=1269 PGID=1269 started: sleep 8
job-shell> jobs

ID      PID      PGID      COMMAND      STATE
-----
1       1267      1267      sleep 15      RUNNING
2       1269      1269      sleep 8        RUNNING
job-shell> update
Job states updated.
job-shell> jobs

ID      PID      PGID      COMMAND      STATE
-----
1       1267      1267      sleep 15      COMPLETED
2       1269      1269      sleep 8        COMPLETED
job-shell> clean
Removing completed Job [1] (PID 1267)
Removing completed Job [2] (PID 1269)
job-shell> jobs

ID      PID      PGID      COMMAND      STATE
-----
No active jobs.

job-shell> exit
Job table system terminated.

```

Report: The program implements a job table for background processes. It stores Job ID, PID, PGID, command, and execution state. It monitors processes using non-blocking waitpid(), updates their states, removes completed records, and validates the stored job information.